

/opsx:propose add-feature-scope-analyzer

I'm checking the repository's OpenSpec structure and any existing change/spec conventions so I can propose the new feature scope analyzer cleanly.

Considered feature proposal needs and evaluated file paths

I confirmed the OpenSpec directory exists. I'm now reading the current configuration and any existing change/spec content so the proposal matches the repository's format.

Reviewed 4 files and searched for regex patterns

I found the OpenSpec proposal workflow and I'm reading its instructions now so I can generate the right artifacts rather than guessing the format.

Reviewed SKILL.md and considered command usage

I've got the proposal workflow pinned down. Next I'm checking whether the OpenSpec CLI is available in this workspace so I can create the change and generate the artifacts directly.

Ran `openspec --version`

Determined change status and ran command

Ran `openspec status --change "add-feature-scope-analyz..."` ▾

◉ `openspec status --change "add-feature-scope-analyzer" --json`

ng `"add-feature-scope-analyzer" --json`

✗ Error: Change 'add-feature-scope-analyzer' not found. No changes exist. Create one with: `openspec new change <name>`

Created artifacts and changes

Ran `openspec new change "add-feature-scope-analyzer"`

Inspected status and ran command in terminal

Ran `openspec status --change "add-feature-scope-analyz...`

Generating a proposal

The scaffold is in place. I'm now pulling the artifact instructions and writing the proposal first so the downstream design and task artifacts can build on it.

Ran `openspec instructions proposal --change "add-featu...`

Decided on project content and reviewed design instructions

Ran `openspec instructions design --change "add-feature...`

Ran `openspec instructions specs --change "add-feature-...`

Ran `openspec instructions tasks --change "add-feature-...`

Planned feature scope analysis changes and file creation strategy

I'm creating the proposal, design, spec, and task artifacts now so the change is complete and ready to review.

Created 4 files

The change artifacts are written. I'm verifying the OpenSpec status now so I can confirm the proposal is complete and ready for implementation.

Ran `openspec status --change "add-feature-scope-analyz...`

Clarifying formatting guidelines

 **Proposal completed**

The OpenSpec change for add-feature-scope-analyzer has been created and all required artifacts are complete.

✅ Proposal completed

The OpenSpec change for add-feature-scope-analyzer has been created and all required artifacts are complete.

Created artifacts

- [proposal.md](#)
- [design.md](#)
- [spec.md](#)
- [tasks.md](#)

Verification

I verified the result with:

- `openspec status --change "add-feature-scope-analyzer"`

Evidence: the command reported:

- Progress: 4/4 artifacts complete
- All artifacts complete!

Next step

If you want, I can now help implement the change or refine the scope analysis format.



The current OpenSpec proposal treats the feature scope analyzer as a documentation capability instead of a React application.

Please refine the OpenSpec artifacts so the change represents a React-based web application called “Feature Scope Analyzer”.

The application should:

- allow users to enter a project idea in a textarea
- detect predefined technical keywords such as AI, blockchain, realtime, payments, chat, and video streaming
- calculate project complexity based on detected keyword count
- display Low, Medium, or High complexity levels
- provide MVP recommendations dynamically
- update results in the UI after clicking an Analyze button

[Click to Edit](#)

Please update:

- proposal.md
- design.md
- spec.md
- tasks.md

The updated artifacts should include:

- frontend behavior
- complexity evaluation rules
- UI rendering expectations
- validation for empty input
- implementation tasks for React components and analysis logic
- test and verification tasks

Optimized tool selection

I'm updating the OpenSpec artifacts to shift the scope from documentation to a React UI and I'll verify the change status after editing.

Planned file modifications and reviewed 4 files

I've rewritten the artifacts to describe a React application, including the UI interaction, analysis rules, validation behavior, and verification tasks.

Updated proposal.md and checked design files

I've applied the refinements and I'm doing a final verification pass on the updated artifacts and status so the change is consistent and complete.

Reviewed 3 files and ran command

Ran `openspec status --change "add-feature-scope-analyz...`



OpenSpec artifacts updated for a React app

The artifacts now describe a React-based **Feature Scope Analyzer** web application instead of a documentation-only capability.

Updated files

- ✦ proposal.md
- ✦ design.md
- ✦ spec.md
- ✦ tasks.md

Included in the refinement

- frontend behavior for a React UI
- detection of predefined technical keywords
- complexity evaluation rules

ADD FEATURE SCOPE ANALYZER

- frontend behavior for a React UI
- detection of predefined technical keywords
- complexity evaluation rules
- UI rendering expectations
- empty input validation
- implementation tasks for React components and analysis logic
- test and verification tasks

Verification

I verified the result with:

- `openspec status --change "add-feature-scope-analyzer"`

Evidence:

- Progress: `4/4 artifacts complete`
- `All artifacts complete!`

If you want, I can next help convert this into a concrete React implementation plan or start building the app.

Restore Checkpoint · 

`/opsx:apply` add-feature-scope-analyzer

 spec.md:1-86

I'm checking the current change status and the repository context before applying the change, so I can follow the OpenSpec apply workflow and avoid missing dependencies.

Reviewed SKILL.md and ran command

Ran `openspec status --change "add-feature-scope-analyz...`

Ran `openspec instructions apply --change "add-feature-...`

Inspecting the React app

I've confirmed the change is ready to apply. I'm checking the repository for an existing React app or test setup so I can implement the analyzer in the right place and keep the work aligned with the current project structure.

Searched for files matching package.json and created React app

I confirmed the repository currently has no application code, so I'm setting up the smallest React project that can host the Feature Scope Analyzer and support the test coverage you asked for.

Ran `node -v ; npm -v`

Planned React app creation

Ran `npm create vite@latest feature-scope-analyzer -- -...`

Considering installation options

The Vite scaffold is waiting on a confirmation prompt. I'm choosing the non-auto-start path so I can control the dependency installation and test setup explicitly.

Sent `n` to terminal



Lark (Preview) • 1x

`npm create vite@latest feature-scope-analyzer -- --template react` completed

Inspecting terminal outputs

The React scaffold is in place. I'm reading the generated files now so I can wire in the analyzer logic, add tests, and make the UI match the spec precisely.

Reviewed 5 files and considered app testing setup

I've confirmed the starter app is generic Vite content. I'm installing the test tooling next so I can implement the analyzer with real verification rather than only a visual pass.

Ran `cd feature-scope-analyzer ; npm install ; npm inst...`

Planned testing setup and created helper functions

I've pinned down the app structure and I'm adding the analyzer logic and test coverage now so the UI behavior is driven by predictable, verifiable rules.

Created 4 files and ran tests

Ran `cd feature-scope-analyzer ; npx vitest run` ▾

⦿ `cd feature-scope-analyzer ; npx vitest run`

—[3/3]—

```
Test Files  1 failed (1)
Tests      3 failed (3)
Start at   21:51:20
Duration   3.57s (transform 186ms, setup 229ms, import 456ms, tests 452ms, env
ironment 1.97s)
```

Evaluating terminal output access

The tests are failing as expected against the starter UI. I'm now replacing the placeholder app with the analyzer behavior and updating the test command so the verification loop is fully wired.

Read `content.txt`

Modified keyword detection and enhanced display strategy

Updated analyzer.js

The analyzer UI and logic are in place. I'm running the test suite and build now to confirm the behavior and make sure the React app compiles cleanly.

Ran `cd feature-scope-analyzer ; npm test ; npm run bui...` 

❶ `cd feature-scope-analyzer ; npm test ; npm run build`

```
hop-project/feature-scope-analyzer/node_modules/vite/dist/node/chunks/node.js:33
559:19)
    at async Object.buildApp (file:///C:/Users/Harini%20Venkatesh/Desktop/sdd-workshop-project/feature-scope-analyzer/node_modules/vite/dist/node/chunks/node.js:33556:153)
    at async CAC.<anonymous> (file:///C:/Users/Harini%20Venkatesh/Desktop/sdd-workshop-project/feature-scope-analyzer/node_modules/vite/dist/node/cli.js:777:3)
  {
    errors: [Getter/Setter]
  }
```

The first verification run surfaced a small syntax issue in the analyzer helper, so I'm fixing that now and rerunning the test/build cycle.

Reviewed and updated analyzer.js

Ran `cd feature-scope-analyzer ; npm test ; npm run bui...`

Adjusted test strategies and modified test code

One test is still too broad because it matches the textarea text as well as the rendered result. I'm tightening that assertion to target the results panel specifically so the verification reflects the UI behavior you asked for.

Updated App.test.jsx and ran command

Ran `cd feature-scope-analyzer ; npm test ; npm run bui...`

Reviewed and updated App.test.jsx

Ran `cd feature-scope-analyzer ; npm test ; npm run bui...`

Updated task status and checked completion status

The implementation and verification are passing. I'm syncing the task checklist now so the OpenSpec change reflects the completed work and the apply phase is fully closed out.

Updated tasks.md and verified status

Ran `cd /d C:\Users\Harini Venkatesh\Desktop\sdd-worksh...` ▾

```
❶ cd /d C:\Users\Harini Venkatesh\Desktop\sdd-workshop-project ; openspec status --change "add-feature-scope-analyzer"
```

```
+ cd /d C:\Users\Harini Venkatesh\Desktop\sdd-workshop-project ; opensp ...
```

```
+ ~~~~~
```

```
+ CategoryInfo          : InvalidArgument: (:) [Set-Location], ParameterBi  
ndingException
```

```
+ FullyQualifiedErrorId : PositionalParameterNotFound,Microsoft.PowerShell  
.Commands.SetLocationCommand
```

```
✗ Error: Change 'add-feature-scope-analyzer' not found. No changes exist. Create  
one with: openspec new change <name>
```

Running a command in terminal

The last status check hit a PowerShell path quoting issue, so I'm rerunning it from the repo root explicitly to get a clean verification signal.

Ran `cd 'C:\Users\Harini Venkatesh\Desktop\sdd-workshop...`

✅ Feature Scope Analyzer applied

The change is now implemented as a React-based web app, and the OpenSpec tasks are marked complete.

What was added

- React app scaffold and analyzer UI in `App.jsx`
- Analysis logic and keyword rules in `analyzer.js`
- UI styling in `App.css` and `index.css`
- Test coverage in `App.test.jsx`
- Test script added in `package.json`
- OpenSpec task checklist updated in `tasks.md`

Verified

I verified the implementation with fresh commands and evidence:

1. `cd feature-scope-analyzer && npm test && npm run build`
 - **Test result:** 1 passed (1) test file, 3 passed (3) tests
 - **Build result:** ✓ built in 247ms
2. `openspec status --change "add-feature-scope-analyzer"`
 - **Result:** Progress: 4/4 artifacts complete
 - **Status:** all artifacts complete

Notes

The app currently supports:

- textarea input

- keyword detection for AI, blockchain, realtime, payments, chat, and video streaming
- complexity evaluation: Low / Medium / High
- dynamic MVP recommendations
- empty-input validation
- UI updates only after clicking **Analyze**

If you want, I can also start the dev server so you can view the app locally.

Restore Checkpoint · 

The keyword detection logic needs refinement.

Current issues:

- "ai" is being incorrectly matched inside unrelated words
- "payment" is not detected while "payments" is detected

Please improve the keyword detection so:

- keywords are matched as whole words or intended phrases only
- partial substring matches do not trigger false positives
- both singular and plural forms are supported where appropriate

Examples:

- "chair" should not detect "ai"
- "payment" and "payments" should both work
- "video streaming" should still work as a phrase

Please also update the tests to verify:

- no false positives from substring matches

- no false positives from substring matches
- singular/plural keyword detection
- phrase detection behavior

I'm checking the current analyzer and test files so I can refine the matching logic against the actual implementation and update the tests to capture the false-positive cases you described.

Reviewed 2 files and updated keyword detection logic

I've confirmed the current implementation uses raw substring matching. I'm switching it to boundary-based patterns and adding targeted logic tests for false positives, singular/plural handling, and phrase matching.

Updated App.test.jsx and analyzer.js, reviewed 2 files

The matching logic is updated. I'm running the test and build checks again now to confirm the false-positive cases and the UI behavior both pass with the refined keyword rules.

Ran `cd feature-scope-analyzer ; npm test ; npm run bui...`

Adjusted detection tests and refined test logic

The logic is passing, and the remaining failure is just the test expecting a different keyword order than the current UI renders. I'm aligning the assertion with the actual rendered sequence and rerunning the verification.

Compacted conversation

Edited `App.test.jsx` +3 -1

Ran `cd feature-scope-analyzer ; npm test ; npm run bui...`

Clarifying formatting rules

Edited `App.test.jsx` +3 -1

Ran `cd feature-scope-analyzer ; npm test ; npm run bui...`

Clarifying formatting rules

✅ Update complete

The keyword detection logic and tests are now aligned and verified.

What changed

- Refined matching in `js analyzer.js` to use **whole-word / whole-phrase regex checks**
- Added support for **singular/plural matching** via `payments?`
- Updated the UI test assertion in `App.test.jsx` to match the rendered keyword order

Verification

I verified this with a fresh run of:

- `cd feature-scope-analyzer && npm test && npm run build`

Evidence:

- **Tests:** 1 passed (1) suite, 6 passed (6) tests
- **Build:** ✓ built in 264ms

Notes

There was a transient PowerShell `Set-Location` warning from the terminal wrapper, but the command still completed successfully and both verification steps passed.

Good. This is the highest-value improvement left.

Open your keyword detection file (`analyzer.js` probably) and replace the simple keyword array/object with this:

```
const keywordGroups = {
  ai: ["ai", "artificial intelligence", "machine learning", "ml"],
  blockchain: ["blockchain", "block chain", "crypto", "web3"],
  realtime: ["realtime", "real-time", "live", "instant"],
  payments: ["payment", "payments", "banking", "transactions"],
  chat: ["chat", "chatbot", "messaging", "messages"],
  "video streaming": [
    "video streaming",
    "livestream",
    "live stream",
    "live video",
    "streaming",
  ],
}
```

Then update detection logic to search ALL aliases.

Click to Edit

Use something like:

```
const detectedKeywords = []

Object.entries(keywordGroups).forEach(([mainKeyword, aliases]) => {
  const found = aliases.some((term) =>
    normalizedInput.includes(term)
  )
})
```

```
if (found) {  
  detectedKeywords.push(mainKeyword)  
}  
})
```

Also make sure you normalize input first:

```
const normalizedInput = input.toLowerCase()
```

After that test these:

- block chain app
- livestream classes
- chatbot ecommerce assistant
- crypto donation tracker
- live driver tracking
- banking platform
- instant messaging app

If this works, your app suddenly feels much smarter without adding complexity.

Optimized tool selection

I'm opening the analyzer and its tests to update the alias-based matching and verify the new cases end-to-end.

Reviewed analyzer.js and App.test.jsx, updated files

I've confirmed the current analyzer is still using regex-based matching. I'm swapping in alias groups now and adding focused tests for the new phrases you listed.

Reviewed App.jsx and considered alias matching

I've added the new alias-focused expectations first so we can confirm the current implementation misses them before changing the matcher.

Updated App.test.jsx

Ran `cd feature-scope-analyzer ; npm test` ▾

❶ `cd feature-scope-analyzer ; npm test`

—————[1/1]—

```
Test Files  1 failed (1)
  Tests     1 failed | 6 passed (7)
  Start at  10:56:31
  Duration  5.86s (transform 176ms, setup 355ms, import 506ms, tests 2.08s, env
ironment 2.54s)
```

Implementing modifications

The failure is reproduced: the current matcher is still missing `block chain`, `livestream`, and the other alias phrases. I'm replacing the detection with grouped alias matching that preserves the earlier false-positive protections.

Updated analyzer.js and verified regex escape patterns

Ran `cd feature-scope-analyzer ; npm test` ▾

❶ `cd feature-scope-analyzer ; npm test`

—————[1/1]—

```
Test Files 1 failed (1)
Tests      1 failed | 6 passed (7)
Start at   10:56:59
Duration   4.63s (transform 125ms, setup 183ms, import 327ms, tests 2.06s, env
ironment 1.69s)
```

Updated test expectations and evaluated application tests

The new matching is working for the alias cases. I'm updating the remaining phrase assertion to reflect that the broader `live` alias legitimately also matches `realtime`.

Edited `App.test.jsx` +1 -1

Ran `cd feature-scope-analyzer ; npm test ; npm run bui...`

✓ Alias-based keyword detection is in place

I updated the analyzer in `analyzer.js` to use grouped aliases and scan all of them against normalized input, while still keeping boundary-aware matching so unrelated words do not trigger false positives.

I also updated the tests in `App.test.jsx` to cover the new cases you listed.

What now matches

- block chain app → blockchain
- livestream classes → video streaming
- chatbot ecommerce assistant → chat
- crypto donation tracker → blockchain
- live driver tracking → realtime
- banking platform → payments

✅ Alias-based keyword detection is in place

I updated the analyzer in `analyzer.js` to use grouped aliases and scan all of them against normalized input, while still keeping boundary-aware matching so unrelated words do not trigger false positives.

I also updated the tests in `App.test.jsx` to cover the new cases you listed.

What now matches

- block chain app → blockchain
- livestream classes → video streaming
- chatbot ecommerce assistant → chat
- crypto donation tracker → blockchain
- live driver tracking → realtime
- banking platform → payments
- instant messaging app → realtime and chat

Verification

I verified the change with a fresh run of:

- `cd feature-scope-analyzer && npm test && npm run build`

Evidence:

- Tests: 1 passed suite, 7 passed tests
- Build: Vite production build completed successfully

If you want, I can also make the rendered keyword labels more polished so the UI shows AI and other categories in title case instead of lowercase.